Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Teoria de Sistemas 1

Sección: A

Docente: Ivan Alejandro Esteban Archila

“Manual Tecnico Clinica Dental”

Estudiantes:

Luis Alejandro Regalado Villatoro - 202131920
Carlos Alfredo López de León - 202231051
Mynor Miguel Monzón Martínez - 202230884
Herson Isaías Aguilar Juárez - 202230685
María Yohana Pérez López - 202030086

Quetzaltenango, 29 de octubre de

2026

Introducción

Este manual está dirigido al personal técnico que instala, mantiene, despliega o modifica el Sistema de gestión para clínica dental: desarrolladores, administradores del servidor y evaluadores del proyecto. Complementa al Manual de Usuario, que explica el uso del sistema desde la interfaz; aquí se describe cómo está construido, cómo se configura y cómo se opera.

El sistema es una aplicación web cliente-servidor. El frontend en Angular se sirve con nginx, que además reenvía las peticiones con prefijo /api al backend en Spring Boot. El backend expone una API REST protegida con JWT y guarda la información en PostgreSQL. Los tres servicios corren en contenedores Docker orquestados con Docker Compose, tanto en desarrollo como en la instancia AWS EC2 de producción, y el despliegue se automatiza con GitHub Actions.

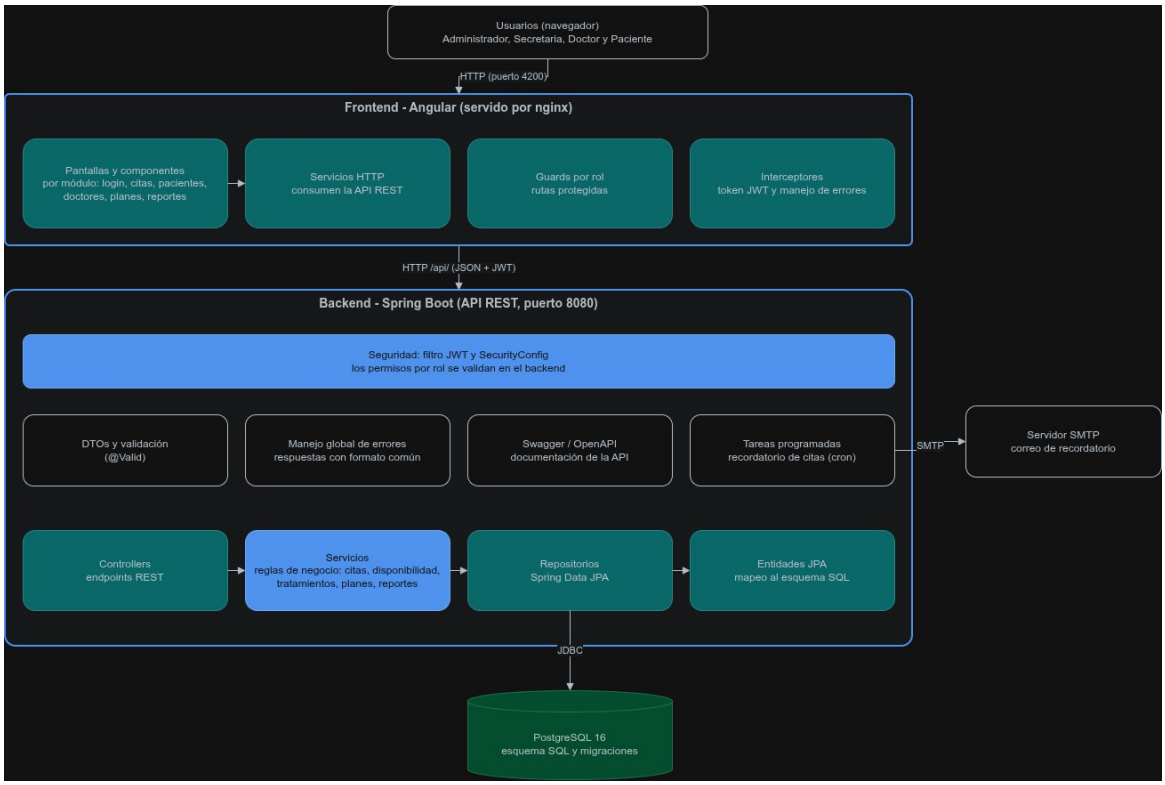
Funcionalmente cubre usuarios y roles, pacientes con lista negra e historial clínico, doctores con horarios y bloqueos, agendamiento y reprogramación de citas sin solapes, control de inasistencias, tratamientos por cita con catálogo, planes de tratamiento con abonos, recordatorios por correo y reportes. Todos los valores de este manual se tomaron del código de la rama develop y del archivo de información de la instancia EC2.

1. Arquitectura del sistema

1.1 Arquitectura cliente-servidor

Componente	Tecnología	Responsabilidad
Cliente	Navegador web con la aplicación Angular	Interfaz por rol; guarda el token JWT y lo envía en cada petición.
Servidor web	nginx (contenedor frontend, puerto 80, publicado en 4200)	Sirve los archivos estáticos de Angular y reenvía /api/ al backend (nginx.conf).
Servidor de aplicación	Spring Boot (contenedor backend, puerto 8080)	API REST, seguridad, reglas de negocio y envío de correos programados.
Base de datos	PostgreSQL 16 (contenedor db, puerto 5432 solo en 127.0.0.1)	Persistencia en el volumen postgres_data.

nginx reenvía /api/ a http://backend:8080/ y agrega las cabeceras X-Forwarded-*, incluida X-Forwarded-Prefix /api; el backend las interpreta con server.forward-headers-strategy=framework para que Swagger genere rutas correctas detrás del proxy. En desarrollo, proxy.conf.json cumple la misma función y envía /api a http://localhost:8080.



1.2 Arquitectura en capas del backend

El código está en backend/src/main/java/com/teo1/clinicadental/ y se organiza en capas:

Capa	Paquete	Responsabilidad
Presentación (API)	controller	14 controllers REST; reciben DTO validados con @Valid y documentan cada operación con springdoc (@Tag, @Operation).
Contratos	dto	Objetos de entrada y salida (Request/Response); las entidades no se exponen.
Negocio	service	Reglas de negocio y transacciones (CitaService, DisponibilidadCitaService, PlanTratamientoService, etc.). CitaMapper convierte entidades a DTO.
Acceso a datos	repository	17 repositorios Spring Data JPA; consultas derivadas y JPQL con parámetros, proyecciones para reportes y bloqueos pesimistas.
Dominio	model	Entidades JPA y enumeraciones (EstadoCita, EstadoPlan, Rol, DiaSemana, Turno...).
Seguridad	security	JwtService (firma y validación) y JwtAuthenticationFilter.

Capa	Paquete	Responsabilidad
Configuración	config	SecurityConfig, OpenApiConfig, ClockConfig (zona horaria) y AdminInitializer (primer administrador).
Errores	exception	GlobalExceptionHandler: respuestas JSON uniformes con status, error, mensaje y path.

2. Tecnologías y herramientas

Componente	Tecnología	Versión
Lenguaje backend	Java (JDK)	21 (pom.xml java.version)
Framework backend	Spring Boot (starter parent)	4.1.1
Módulos Spring	web, data-jpa, security, validation, mail	Gestionadas por Spring Boot 4.1.1
Documentación API	springdoc-openapi-starter-webmvc-ui	3.1.1
Tokens	jjwt (api, impl, jackson)	0.13.0
Utilidades	Lombok	Gestionada por Spring Boot
Construcción backend	Maven Wrapper / Maven	Wrapper 3.3.4 / Maven 3.9.16
Imagen de construcción backend	maven	3.9-eclipse-temurin-21
Imagen de ejecución backend	eclipse-temurin	21-jre-alpine
Framework frontend	Angular (@angular/core)	^22.1.0 (22.1.5 en package-lock.json)
CLI frontend	@angular/cli, @angular/build	^22.1.6
Lenguaje frontend	TypeScript	~6.0.2 (6.0.3 instalado)
Estilos	Bootstrap / Bootstrap Icons	5.3.8 / 1.13.1
Programación reactiva	RxJS	~7.8.0
Decodificación JWT en cliente	jwt-decode	^4.0.0
Pruebas frontend	Vitest con jsdom	^4.0.8 / ^28.0.0
Entorno Node	Node.js / npm	24.15.0 / npm@11.19.0

Componente	Tecnología	Versión
Imagen de construcción frontend	node	24.15.0-alpine
Servidor web	nginx	stable-alpine
Base de datos	PostgreSQL	16 (postgres:16-alpine)
Contenedores	Docker y Docker Compose	EC2: Docker 25.0.14, Compose v5.5.0
CI/CD	GitHub Actions	checkout@v6, setup-java@v6, setup-node@v6, docker/login-action@v4, docker/build-push-action@v7, appleboy/ssh-action@v1.2.0
Registro de imágenes	GitHub Container Registry (ghcr.io)	Etiqueta = SHA del commit
Nube	AWS EC2	t3.micro, Amazon Linux 2023

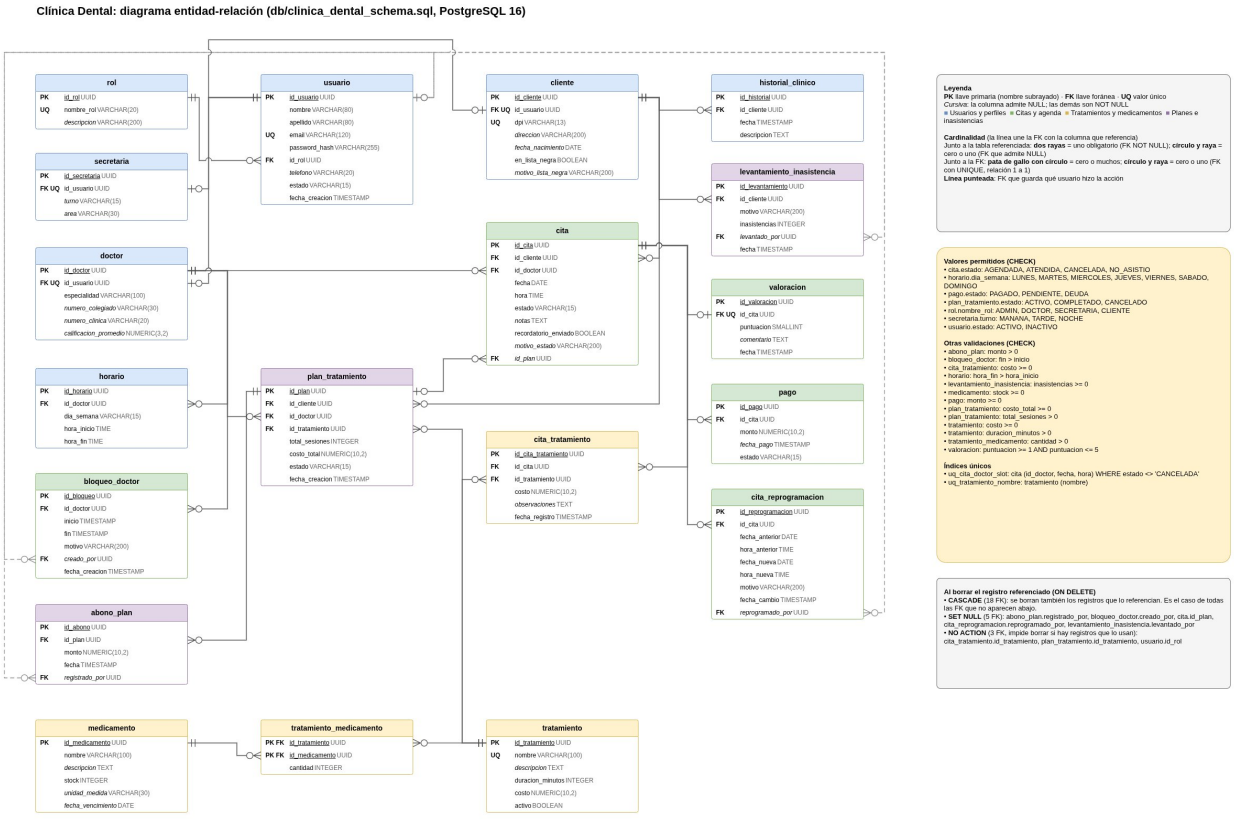
3. Estructura del repositorio

```
TEO1_ClinicaDental/
|-- .github/workflows/        ci.yml y cd.yml
|-- backend/                  API Spring Boot (pom.xml, mvnw, Dockerfile)
|   |-- src/main/java/com/teo1/clinicadental/   codigo por capas
|   |-- src/main/resources/application.properties
|   `-- src/test/java/        pruebas unitarias y de integracion
|-- clinica-dental-frontend/  aplicacion Angular (Dockerfile, nginx.conf,
proxy.conf.json)
|   `-- src/app/              componentes, servicios, guards e interceptores
|-- db/
|   |-- clinica_dental_schema.sql   esquema completo para una base vacia
|   |-- migrations/           cambios para bases con datos (aplicacion
manual)
|   `-- demo/datos-demo.sql   datos de demostracion
|-- docs/                     respaldo y restauracion, revision OpenAPI
|   `-- diagramas/            ER, despliegue y CI/CD (.drawio y .png)
|-- scripts/                  deploy-ec2.sh, respaldar-db.sh, restaurar-
db.sh
|-- docker-compose.yml        servicios db, backend y frontend
|-- .env.example              plantilla de variables sin secretos
`-- README.md
```

IMPORTANTE: el archivo .env con los valores reales no se versiona: está en .gitignore (línea 18) y solo existe en cada equipo y en el servidor.

4. Modelo de datos

El esquema completo está en db/clinica_dental_schema.sql. Todas las llaves primarias son UUID generadas con gen_random_uuid() (extensión pgcrypto). El archivo empieza con DROP TABLE, por lo que solo se usa en bases nuevas; Docker Compose lo carga automáticamente la primera vez que se crea el volumen.



4.1 Diccionario de datos

Tabla rol

Columna	Tipo	Restricciones
id_rol	UUID	PK
nombre_rol	VARCHAR(20)	NOT NULL, UNIQUE, CHECK ADMIN, DOCTOR, SECRETARIA, CLIENTE
descripcion	VARCHAR(200)	

Tabla usuario

Columna	Tipo	Restricciones
id_usuario	UUID	PK

Columna	Tipo	Restricciones
nombre / apellido	VARCHAR(80)	NOT NULL
email	VARCHAR(120)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL (BCrypt)
id_rol	UUID	NOT NULL, FK rol
telefono	VARCHAR(20)	
estado	VARCHAR(15)	NOT NULL, DEFAULT ACTIVO, CHECK ACTIVO o INACTIVO
fecha_creacion	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabla cliente

Columna	Tipo	Restricciones
id_cliente	UUID	PK
id_usuario	UUID	NOT NULL, UNIQUE, FK usuario ON DELETE CASCADE
dpi	VARCHAR(13)	NOT NULL, UNIQUE
direccion	VARCHAR(200)	
fecha_nacimiento	DATE	
en_lista_negra	BOOLEAN	NOT NULL, DEFAULT FALSE
motivo_lista_negra	VARCHAR(200)	

Tabla doctor

Columna	Tipo	Restricciones
id_doctor	UUID	PK
id_usuario	UUID	NOT NULL, UNIQUE, FK usuario ON DELETE CASCADE
especialidad	VARCHAR(100)	NOT NULL
numero_colegiado	VARCHAR(30)	
numero_clinica	VARCHAR(20)	
calificacion_promedio	NUMERIC(3,2)	DEFAULT 0

Tabla secretaria

Columna	Tipo	Restricciones
id_secretaria	UUID	PK
id_usuario	UUID	NOT NULL, UNIQUE, FK usuario
turno	VARCHAR(15)	CHECK MANANA, TARDE, NOCHE
area	VARCHAR(30)	DEFAULT RECEPCION

Tabla historial_clinico

Columna	Tipo	Restricciones
id_historial	UUID	PK
id_cliente	UUID	NOT NULL, FK cliente
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
descripcion	TEXT	NOT NULL

Tabla horario

Columna	Tipo	Restricciones
id_horario	UUID	PK
id_doctor	UUID	NOT NULL, FK doctor
dia_semana	VARCHAR(15)	NOT NULL, CHECK LUNES a DOMINGO
hora_inicio / hora_fin	TIME	NOT NULL, CHECK hora_fin > hora_inicio

Tabla tratamiento

Columna	Tipo	Restricciones
id_tratamiento	UUID	PK
nombre	VARCHAR(100)	NOT NULL, índice único uq_tratamiento_nombre
descripcion	TEXT	
duracion_minutos	INTEGER	NOT NULL, CHECK > 0
costo	NUMERIC(10,2)	NOT NULL, CHECK >= 0

Columna	Tipo	Restricciones
activo	BOOLEAN	NOT NULL, DEFAULT TRUE

Tabla plan_tratamiento

Columna	Tipo	Restricciones
id_plan	UUID	PK
id_cliente / id_doctor	UUID	NOT NULL, FK
id_tratamiento	UUID	NOT NULL, FK tratamiento
total_sesiones	INTEGER	NOT NULL, CHECK > 0
costo_total	NUMERIC(10,2)	NOT NULL, CHECK >= 0
estado	VARCHAR(15)	NOT NULL, DEFAULT ACTIVO, CHECK ACTIVO, COMPLETADO, CANCELADO
fecha_creacion	TIMESTAMP	NOT NULL

Tabla cita

Columna	Tipo	Restricciones
id_cita	UUID	PK
id_cliente / id_doctor	UUID	NOT NULL, FK
fecha	DATE	NOT NULL
hora	TIME	NOT NULL
estado	VARCHAR(15)	NOT NULL, DEFAULT AGENDADA, CHECK AGENDADA, ATENDIDA, CANCELADA, NO_ASISTIO
notas	TEXT	
recordatorio_enviado	BOOLEAN	NOT NULL, DEFAULT FALSE
motivo_estado	VARCHAR(200)	
id_plan	UUID	FK plan_tratamiento ON DELETE SET NULL
(índice)	uq_cita_doctor_slot	UNIQUE (id_doctor, fecha, hora) WHERE estado <> CANCELADA

Tabla cita_tratamiento

Columna	Tipo	Restricciones
id_cita_tratamiento	UUID	PK
id_cita	UUID	NOT NULL, FK cita
id_tratamiento	UUID	NOT NULL, FK tratamiento
costo	NUMERIC(10,2)	NOT NULL, CHECK >= 0
observaciones	TEXT	
fecha_registro	TIMESTAMP	NOT NULL

Tabla cita_reprogramacion

Columna	Tipo	Restricciones
id_reprogramacion	UUID	PK
id_cita	UUID	NOT NULL, FK cita
fecha_anterior / hora_anterior	DATE / TIME	NOT NULL
fecha_nueva / hora_nueva	DATE / TIME	NOT NULL
motivo	VARCHAR(200)	NOT NULL
fecha_cambio	TIMESTAMP	NOT NULL
reprogramado_por	UUID	FK usuario ON DELETE SET NULL

Tabla bloqueo_doctor

Columna	Tipo	Restricciones
id_bloqueo	UUID	PK
id_doctor	UUID	NOT NULL, FK doctor
inicio / fin	TIMESTAMP	NOT NULL, CHECK fin > inicio
motivo	VARCHAR(200)	NOT NULL
creado_por	UUID	FK usuario
fecha_creacion	TIMESTAMP	NOT NULL

Tabla levantamiento_inasistencia

Columna	Tipo	Restricciones
id_levantamiento	UUID	PK
id_cliente	UUID	NOT NULL, FK cliente
motivo	VARCHAR(200)	NOT NULL
inasistencias	INTEGER	NOT NULL, CHECK >= 0
levantado_por	UUID	FK usuario
fecha	TIMESTAMP	NOT NULL

Tabla abono_plan

Columna	Tipo	Restricciones
id_abono	UUID	PK
id_plan	UUID	NOT NULL, FK plan_tratamiento
monto	NUMERIC(10,2)	NOT NULL, CHECK > 0
fecha	TIMESTAMP	NOT NULL
registrado_por	UUID	FK usuario

El esquema también crea las tablas medicamento, tratamiento_medicamento, valoracion y pago, que no tienen entidad JPA ni pantalla en la versión actual. Índices adicionales: idx_cita_cliente, idx_cita_doctor, idx_cita_fecha, idx_usuario_rol, idx_horario_doctor, idx_reprog_cita, idx_historial_cliente, idx_cita_tratamiento_cita, idx_bloqueo_doctor (id_doctor, inicio), idx_cita_plan, idx_plan_cliente, idx_plan_doctor, idx_abono_plan e idx_levantamiento_cliente (id_cliente, fecha). El esquema inserta los cuatro roles y un catálogo inicial de tratamientos si la tabla está vacía.

4.2 Migraciones manuales

Las bases con datos (como la de EC2) no se recrean: se actualizan con los archivos de db/migrations/, que se pueden ejecutar varias veces sin error y no borran datos. Cada migración corre en una transacción. Se aplican en este orden:

Orden	Archivo	Para qué sirve	Dependencias
1	2026-10-sprint4-cita.sql	Estados nuevos de la cita (COMPLETADA pasa a	Tabla cita del esquema base.

Orden	Archivo	Para qué sirve	Dependencias
		ATENDIDA, INASISTENCIA a NO_ASISTIO, APLAZADA a AGENDADA), recordatorio_enviado y el índice parcial uq_cita_doctor_slot.	
2	2026-10-sprint5.sql	motivo_estado en cita, tabla bloqueo_doctor, nueva forma de cita_tratamiento, motivo y reprogramado_por en cita_reprogramacion, uq_tratamiento_nombre y catálogo inicial.	Va después de 2026-10-sprint4-cita.sql; usa doctor, usuario, cita y tratamiento.
3	2026-10-sprint6-tratamientos.sql	Agrega la columna tratamiento.activo y carga el catálogo de 20 tratamientos (10 nuevos sobre los 10 de 2026-10-sprint5.sql).	Tabla tratamiento (existente antes del Sprint 6).
4	2026-10-sprint6-planes.sql	Crea plan_tratamiento y abono_plan y agrega cita.id_plan (HU-26).	cliente, doctor, tratamiento, cita y usuario (anteriores al Sprint 6).
5	2026-10-sprint6-inasistencias.sql	Crea levantamiento_inasistencia (HU-25).	cliente y usuario.

Las tres migraciones del Sprint 6 van después de 2026-10-sprint5.sql, como indica el encabezado de cada archivo. Se aplicaron en EC2 el 28 de octubre de 2026 en el orden de la tabla (issue #213, según informó Alejo). El orden entre estas tres no importa porque ninguna usa algo que cree otra (se revisó leyendo el código), pero la única corrida real fue en el orden de la tabla; no se probó el orden inverso.

- Se respalda la base antes de migrar con scripts/respaldar-db.sh.
- Se aplican las tres antes de levantar el backend nuevo, porque ddl-auto=validate impide arrancar si falta una tabla o columna.
- db/demo/datos-demo.sql va al final, porque busca los tratamientos del catálogo por nombre.

El archivo 2026-10-sprint6-tratamientos.sql indica en su encabezado que los nombres, duraciones y precios propuestos requieren aprobación del equipo.

4.3 Datos de demostración

db/demo/datos-demo.sql crea, con ids que empiezan con de000000-, 2 doctores con horarios, 1 secretaria, 5 pacientes (uno en lista negra y una paciente restringida por inasistencias), citas de ayer, hoy y mañana en todos los estados, tratamientos, una reprogramación, historial clínico, un bloqueo, planes y abonos. Cada corrida borra la demo anterior y la vuelve a cargar dentro de una transacción. La contraseña no está en el archivo: se pasa con la variable psql clave_demo y se guarda cifrada con crypt(). Con -v solo_limpiar=1 solo se borra la demo. El procedimiento está en la sección 12.6.

5. Seguridad

5.1 Autenticación con JWT

POST /auth/login valida el correo y la contraseña y devuelve un token JWT firmado con HMAC (Keys.hmacShaKeyFor sobre JWT_SECRET decodificado en base64). El token lleva como subject el id del usuario y el claim del rol. La expiración se configura con app.jwt.expiration-ms (application.properties:16), que toma JWT_EXPIRATION_MS y por defecto vale 86400000 ms (24 horas). JwtAuthenticationFilter lee la cabecera Authorization: Bearer, valida firma y vigencia y comprueba en cada petición que el usuario siga ACTIVO; un usuario desactivado deja de ser autenticado aunque su token no haya vencido. La sesión es STATELESS y CSRF está deshabilitado porque no se usan cookies.

5.2 Cifrado de contraseñas

Las contraseñas se guardan con BCryptPasswordEncoder (SecurityConfig.java:39) en usuario.password_hash. El registro exige al menos 8 caracteres con mayúscula, minúscula, número y carácter especial (RegistroRequest y CrearPersonalRequest).

5.3 Autorización por roles

La autorización se aplica en dos niveles:

- **Por ruta:** SecurityConfig define con requestMatchers qué roles acceden a cada método y ruta (tabla de la sección 7).
- **Por dato en los servicios:** por ejemplo, el CLIENTE y el DOCTOR solo ven sus citas (CitaService), un doctor solo modifica sus horarios (DoctorService) y solo ve sus bloqueos (BloqueoDoctorService), solo el doctor de la cita registra tratamientos (TratamientoService) y un doctor solo crea planes para sí mismo (PlanTratamientoService).

Los errores 401 y 403 se devuelven en JSON con los mensajes Debes iniciar sesion y No tienes permiso para esta accion. En el frontend, rolGuard bloquea las rutas por rol (sección 8.3).

5.4 Validación de entradas y consultas seguras

Los controllers reciben DTO anotados con Jakarta Validation (@NotNull, @NotBlank, @Size, @Pattern, @DecimalMin, @Digits, @Min, @Max) y @Valid. GlobalExceptionHandler responde 400 con el primer campo inválido y el mapa de errores. El acceso a datos usa Spring Data JPA con consultas derivadas y JPQL con parámetros con nombre; no hay consultas nativas armadas con texto, lo que evita la inyección SQL. Las restricciones de la base (CHECK, UNIQUE, FK) son una segunda barrera; su violación se responde con 409.

5.5 CORS y endpoints públicos

CORS permite solo los orígenes de app.cors.allowed-origins (variable CORS_ALLOWED_ORIGINS), los métodos GET, POST, PUT, PATCH, DELETE y OPTIONS, las cabeceras Authorization y Content-Type, con credenciales y una caché de 3600 segundos. En EC2 el despliegue escribe el origen http://<EC2_HOST>:4200. Rutas públicas: POST /auth/registro, POST /auth/login, GET /version, /v3/api-docs/**, /swagger-ui/** y /swagger-ui.html, además de OPTIONS.

5.6 Limitaciones conocidas

Limitación	Ubicación
El token JWT se guarda en localStorage del navegador; un ataque XSS podría leerlo.	auth.service.ts:54; auth.interceptor.ts:13
Swagger UI y el documento OpenAPI son públicos.	SecurityConfig.java:59
spring.jpa.show-sql=true imprime todas las consultas SQL en el log.	application.properties:12
El backend publica el puerto 8080 en todas las interfaces; en EC2 se ve 0.0.0.0:8080 y el grupo de seguridad lo abre a 0.0.0.0/0, además del acceso por nginx.	docker-compose.yml:42; verificado por SSH el 29 de octubre de 2026; consola de AWS, grupo launch-wizard-1
El sistema se sirve por HTTP sin TLS.	README.md:348; cd.yml (CORS http://)
No hay límite de intentos de inicio de sesión.	SecurityConfig.java:60
Existen contraseñas literales de ejemplo en pruebas y documentación (no son de producción).	FlujoCompletoIntegrationTests.java:52; clinica-dental-frontend/src/app/login/README.md:14
Los puertos 80, 443 y 9000 están abiertos a 0.0.0.0/0 en el grupo de seguridad, pero ningún servicio escucha en	consola de AWS, grupo launch-wizard-1; verificado por SSH el 29

Limitación	Ubicación
ellos.	de octubre de 2026
SSH (puerto 22) está abierto a internet (0.0.0.0/0). Se mantiene porque el CD se conecta por SSH desde servidores de GitHub con direcciones variables; se mitiga porque el servidor SSH solo ofrece autenticación por clave pública (y gssapi), sin contraseña.	consola de AWS, grupo launch-wizard-1; verificado por SSH el 29 de octubre de 2026

Se recomienda cerrar en el grupo de seguridad los puertos que no tienen uso (80, 443 y 9000).

6. Lógica de negocio

6.1 Disponibilidad de la cita

DisponibilidadCitaService centraliza la validación de una fecha y hora para un doctor. Se usa al agendar, al reprogramar y al agendar sesiones de un plan. El orden de las reglas es:

Orden	Regla	Código y mensaje
1	La fecha y hora deben ser posteriores a ahora (Clock de Guatemala).	400 La fecha y hora de la cita ya pasaron
2	La cita de app.citas.duracion-minutos (30) debe caber completa en un horario del doctor para ese día y no cruzar la medianoche.	409 La cita no cabe dentro del horario de atencion del doctor
3	No debe cruzarse con un bloqueo del doctor.	409 El doctor no atiende en ese horario: <motivo>
4	No debe solaparse con otra cita no cancelada del doctor.	409 El doctor ya tiene una cita agendada en ese horario

validarAntelacion exige app.citas.antelacion-cancelacion-horas (24) para que un CLIENTE cancele o reprograme (409 Solo puedes <accion> tu cita con al menos 24 horas de anticipacion).

6.2 Agendar una cita

CitaService.crearCita resuelve el paciente (el CLIENTE agenda para sí; SECRETARIA y ADMIN indican idCliente; el DOCTOR recibe 403), bloquea la fila del doctor con DoctorRepository.findByIdForUpdate (PESSIMISTIC_WRITE), verifica que doctor y paciente estén activos, que el paciente no esté en lista negra (403) y que no haya llegado al límite de inasistencias (InasistenciaService, 409), aplica DisponibilidadCitaService y guarda la cita AGENDADA con saveAndFlush.

6.3 Estados, motivo y plazo

CitaService.cambiarEstado acepta ATENDIDA, CANCELADA y NO_ASISTIO; ninguna cita vuelve a AGENDADA y solo cambia una cita AGENDADA. El motivo es obligatorio salvo para ATENDIDA y admite 200 caracteres (EstadoCitaRequest). Permisos: CANCELADA la hace el CLIENTE dueño, SECRETARIA o ADMIN; ATENDIDA y NO_ASISTIO el DOCTOR de la cita, SECRETARIA o ADMIN. El CLIENTE debe respetar el plazo de 24 horas y el DOCTOR solo marca desde la hora de inicio; SECRETARIA y ADMIN no tienen límite de tiempo.

6.4 Reprogramación

ReprogramacionCitaService permite reprogramar solo citas AGENDADA (CLIENTE dueño, SECRETARIA o ADMIN; el DOCTOR recibe 403). Exige motivo (máximo 200), rechaza la misma fecha y hora, aplica el plazo de 24 horas al CLIENTE, bloquea al doctor, valida la disponibilidad excluyendo la propia cita y registra el cambio en cita_reprogramacion con quién lo hizo.

6.5 Bloqueos de doctor

BloqueoDoctorService crea bloqueos (ADMIN y SECRETARIA) con inicio anterior al fin y motivo obligatorio, con la fila del doctor bloqueada; rechaza el bloqueo si hay citas AGENDADA en el periodo (409). El listado completo lo ven ADMIN, SECRETARIA y el propio doctor; BloqueoVigenteController expone /doctores/{idDoctor}/bloqueos-vigentes con los bloqueos que no han terminado, sin datos internos, para cualquier usuario autenticado.

6.6 Inasistencias

InasistenciaService cuenta las citas NO_ASISTIO del paciente desde su último levantamiento. Con app.citas.max-inasistencias (3) o más, el paciente no puede agendar. ADMIN y SECRETARIA registran un levantamiento con motivo en levantamiento_inasistencia, que guarda la cantidad al momento y reinicia el conteo; si el paciente no está restringido responde 409.

6.7 Tratamientos

TratamientoService administra el catálogo (solo ADMIN; nombre único, costo no negativo, activar o desactivar) y registra tratamientos en una cita: solo el DOCTOR de la cita, solo en citas ATENDIDA y con tratamientos activos. Si no se indica costo se copia el del catálogo, de modo que un cambio de precio no altera lo cobrado. Las observaciones admiten 1000 caracteres.

6.8 Planes y abonos

PlanTratamientoService crea planes de 1 a 50 sesiones; si no se indica costo_total se calcula como costo del catálogo por número de sesiones. Una sesión ocupa lugar mientras su cita está AGENDADA o ATENDIDA. El avance es atendidas sobre totales y el plan se informa

COMPLETADO cuando las atendidas alcanzan el total. Agendar sesión (ADMIN y SECRETARIA) bloquea el plan con PESSIMISTIC_WRITE y reutiliza la validación de citas. Los abonos (ADMIN y SECRETARIA) deben ser mayores que cero y no superar el saldo; se registran con el plan bloqueado para evitar que dos abonos simultáneos superen el saldo.

6.9 Recordatorios por campana y por correo

GET /citas/proximas devuelve las citas AGENDADA del usuario en las próximas horas (por defecto 48, de 1 a 72) y alimenta la campana del frontend. RecordatorioCitaService, activado con @EnableScheduling, corre según app.citas.recordatorio-cron (RECORDATORIO_CRON, por defecto 0 0 7 * * *) en la zona America/Guatemala; envía un correo por cada cita AGENDADA que empieza en las próximas 24 horas y la marca con recordatorio_enviado. Si SMTP_HOST o SMTP_FROM están vacíos no envía nada; si un envío falla, la cita queda sin marcar para el siguiente intento.

6.10 Reportes

ReporteCitaService con ReporteCitaRepository y ReporteGananciaRepository genera citas por doctor y estado, horas más demandadas, inasistencias y cancelaciones, ingresos por tratamiento y ganancias por doctor. Usan consultas JPQL agregadas (GROUP BY) con proyecciones y validan que desde no sea posterior a hasta. Ninguna cuenta citas futuras: la condición cita.fecha < hoy o (cita.fecha = hoy y cita.hora <= hora actual) se aplica con la hora del Clock.

6.11 Transacciones y concurrencia

- Los métodos que escriben usan @Transactional y los de lectura @Transactional(readOnly = true).
- **Bloqueo pesimista del doctor:** DoctorRepository.findByIdForUpdate (PESSIMISTIC_WRITE) serializa agendar, reprogramar y crear bloqueos del mismo doctor, para que dos peticiones simultáneas no validen el mismo hueco.
- **Índice único parcial:** uq_cita_doctor_slot (id_doctor, fecha, hora) WHERE estado <> CANCELADA es la última barrera; si se viola, el servicio responde 409 con el mensaje de solape.
- **Planes:** PlanTratamientoRepository.findByIdForUpdate protege sesiones y abonos.
- CitasConcurrencyIntegrationTests y AbonosConcurrencyIntegrationTests prueban estos casos.

7. API REST

La API tiene 56 operaciones documentadas con springdoc. Los roles salen de SecurityConfig; "Autenticado" significa cualquier rol con token válido, y los servicios aplican además las reglas por dato de la sección 5.3.

7.1 Autenticación

Método	Ruta	Roles	Descripción
POST	/auth/registro	Público	Registrar una cuenta de cliente
POST	/auth/login	Público	Iniciar sesión
GET	/auth/me	Autenticado	Obtener el usuario autenticado

7.2 Usuarios

Método	Ruta	Roles	Descripción
GET	/admin/usuarios	ADMIN	Listar usuarios
POST	/admin/usuarios	ADMIN	Crear personal
PUT	/admin/usuarios/{id}	ADMIN	Actualizar un usuario
PATCH	/admin/usuarios/{id}/estado	ADMIN	Cambiar el estado de un usuario
GET	/roles	ADMIN	Listar roles

7.3 Clientes

Método	Ruta	Roles	Descripción
GET	/pacientes	ADMIN, DOCTOR, SECRETARIA	Listar pacientes
POST	/pacientes	ADMIN, SECRETARIA	Crear un paciente
GET	/pacientes/{id}	ADMIN, DOCTOR, SECRETARIA	Obtener un paciente
PUT	/pacientes/{id}	ADMIN, SECRETARIA	Actualizar un paciente
DELETE	/pacientes/{id}	ADMIN, SECRETARIA	Desactivar un paciente
PATCH	/pacientes/{id}/lista-negra	SECRETARIA	Actualizar la condición de lista negra

Método	Ruta	Roles	Descripción
GET	/pacientes/{id}/historial	ADMIN, DOCTOR, SECRETARIA	Listar el historial clínico
POST	/pacientes/{id}/historial	ADMIN, DOCTOR	Agregar una entrada al historial

7.4 Doctores

Método	Ruta	Roles	Descripción
GET	/doctores	Autenticado	Listar doctores
GET	/doctores/{id}	Autenticado	Obtener un doctor
PUT	/doctores/{id}	ADMIN	Actualizar un doctor
DELETE	/doctores/{id}	ADMIN	Desactivar un doctor
GET	/doctores/{id}/horarios	Autenticado	Listar horarios
POST	/doctores/{id}/horarios	ADMIN, DOCTOR, SECRETARIA	Agregar un horario
PUT	/doctores/{id}/horarios/{idHorario}	ADMIN, DOCTOR, SECRETARIA	Actualizar un horario
DELETE	/doctores/{id}/horarios/{idHorario}	ADMIN, DOCTOR, SECRETARIA	Eliminar un horario

7.5 Bloqueos de doctor

Método	Ruta	Roles	Descripción
POST	/doctores/{idDoctor}/bloqueos	ADMIN, SECRETARIA	Crear un bloqueo
GET	/doctores/{idDoctor}/bloqueos	ADMIN, SECRETARIA, DOCTOR (propios)	Listar los bloqueos
DELETE	/doctores/{idDoctor}/bloqueos/{idBloqueo}	ADMIN, SECRETARIA	Eliminar un bloqueo
GET	/doctores/{idDoctor}/bloqueos-vigentes	Autenticado	Listar los bloqueos vigentes

7.6 Citas

Método	Ruta	Roles	Descripción
POST	/citas	CLIENTE, SECRETARIA, ADMIN	Agendar una cita
GET	/citas	Autenticado	Listar citas (filtros estado, fecha, idDoctor, idCliente)
GET	/citas/proximas	Autenticado	Listar citas próximas (horas, por defecto 48)
GET	/citas/{id}	Autenticado	Obtener una cita
PATCH	/citas/{id}/estado	Autenticado	Cambiar el estado de una cita
PATCH	/citas/{id}/reprogramar	Autenticado	Reprogramar una cita

7.7 Tratamientos

Método	Ruta	Roles	Descripción
GET	/tratamientos	ADMIN, DOCTOR, SECRETARIA	Listar el catálogo
POST	/tratamientos	ADMIN	Crear tratamiento del catálogo
PUT	/tratamientos/{id}	ADMIN	Actualizar o desactivar tratamiento
POST	/citas/{idCita}/tratamientos	DOCTOR	Registrar un tratamiento en una cita
GET	/citas/{idCita}/tratamientos	ADMIN, DOCTOR, SECRETARIA	Listar los tratamientos de una cita
GET	/pacientes/{idCliente}/tratamientos	ADMIN, DOCTOR, SECRETARIA	Listar los tratamientos de un paciente
GET	/mis-tratamientos	CLIENTE	Listar mis tratamientos

7.8 Inasistencias

Método	Ruta	Roles	Descripción
GET	/pacientes/{id}/inasistencias	ADMIN, DOCTOR, SECRETARIA	Consultar las inasistencias
POST	/pacientes/{id}/inasistencias/levantamientos	ADMIN, SECRETARIA	Levantar la restricción

7.9 Planes de tratamiento

Método	Ruta	Roles	Descripción
POST	/planes	ADMIN, DOCTOR, SECRETARIA	Crear un plan
GET	/planes	ADMIN, DOCTOR, SECRETARIA	Listar planes
GET	/planes/{id}	ADMIN, DOCTOR, SECRETARIA	Obtener un plan con avance, saldo y citas
POST	/planes/{id}/citas	ADMIN, SECRETARIA	Agendar una sesión del plan
POST	/planes/{id}/abonos	ADMIN, SECRETARIA	Registrar un abono
GET	/planes/{id}/abonos	ADMIN, DOCTOR, SECRETARIA	Consultar abonos y saldo
GET	/mis-planes	CLIENTE	Consultar mis planes

7.10 Reportes

Método	Ruta	Roles	Descripción
GET	/reportes/citas-por-doctor	ADMIN, SECRETARIA	Citas por doctor y estado
GET	/reportes/horas-demandadas	ADMIN, SECRETARIA	Horas más demandadas
GET	/reportes/inasistencias	ADMIN, SECRETARIA	Inasistencias y cancelaciones

Método	Ruta	Roles	Descripción
GET	/reportes/ingresos-por-tratamiento	ADMIN, SECRETARIA	Ingresos por tratamiento
GET	/reportes/ganancias	ADMIN, SECRETARIA	Ganancias por doctor y abonos

7.11 Version

Método	Ruta	Roles	Descripción
GET	/version	Público	Consultar la versión desplegada (commit y fecha de construcción)

7.12 Uso de Swagger

1. Se abre Swagger UI: en local http://localhost:8080/swagger-ui.html o http://localhost:4200/api/swagger-ui.html (Docker); en EC2 http://18.226.37.148:4200/api/swagger-ui/index.html. El JSON está en /v3/api-docs.
2. Se ejecuta POST /auth/login con correo y contraseña y se copia el valor token de la respuesta.
3. Se pulsa Authorize, esquema bearerAuth, se pega solo el token y se confirma.
4. Se ejecutan las operaciones protegidas; el token se envía en la cabecera Authorization.

8. Frontend

8.1 Arquitectura Angular

La aplicación usa componentes standalone (sin NgModules), formularios reactivos y signals (signal, computed, toSignal) para el estado de cada pantalla. app.config.ts registra el router y provideHttpClient(withInterceptors([authInterceptor])). No usa zone.js en sus dependencias. La URL base de la API es environment.apiUrl (/api).

8.2 Carpetas

Carpeta (src/app)	Contenido
core	guards (rolGuard), interceptors (authInterceptor), services (ApiService, AuthService), notificacion (mensajes emergentes) y modelos comunes.

Carpeta (src/app)	Contenido
login, registro, registro-personal	Inicio de sesión, registro de pacientes y gestión de usuarios del personal.
dashboard, header, sidebar, campana	Estructura de la pantalla, menú por rol y campana de citas próximas.
citas	Listado, formulario de agendar y modales de estado y reprogramación (CitaService).
pacientes	Listado, detalle, edición, lista negra e historial clínico (PacienteService).
doctores	Listado y detalle con horarios y bloqueos (DoctorService).
tratamientos-paciente	Catálogo de tratamientos y Mis tratamientos.
planes	Listado, nuevo plan, detalle con sesiones y abonos y Mis planes.
reportes	Pantalla de reportes (ReporteService).

8.3 Rutas y guards por rol

Ruta	Roles
'' (login), registro	Pública
dashboard, citas, doctores, doctores/:id	ADMIN, DOCTOR, SECRETARIA, CLIENTE
admin/usuarios, admin/usuarios/crear, tratamientos	ADMIN
pacientes, pacientes/:id, pacientes/:id/historial	ADMIN, DOCTOR, SECRETARIA
pacientes/crear, pacientes/:id/editar	ADMIN, SECRETARIA
pacientes/:id/lista-negra	SECRETARIA
citas/nueva	CLIENTE, SECRETARIA, ADMIN
planes, planes/nuevo, planes/:id	ADMIN, SECRETARIA, DOCTOR
mis-tratamientos, mis-planes	CLIENTE
reportes	ADMIN, SECRETARIA
(cualquier otra)	Redirige al login

rolGuard(roles) lee el rol del token: si no coincide muestra "No tienes permisos para acceder a esta sección." y navega al Dashboard; sin sesión navega al login. El guard es una ayuda de interfaz: la autorización real la hace el backend.

8.4 Interceptor y servicios HTTP

authInterceptor agrega Authorization: Bearer <token> a cada petición. Ante un 401 fuera de /auth/login y /auth/registro limpia la sesión, muestra "Tu sesión expiró. Inicia sesión nuevamente." y vuelve al login; ante un 403 muestra el mensaje de permisos. AuthService guarda el token, decodifica el rol con jwt-decode, consulta /auth/me y, para CLIENTE y DOCTOR, carga las citas próximas de la campana. Cada módulo tiene su servicio que extiende ApiService y usa HttpClient.

9. Variables de entorno

Variable	Para qué sirve	Valor por defecto	Obligatoria
DB_NAME	Nombre de la base (compose)	clinica_dental	No
DB_USERNAME	Usuario de PostgreSQL	postgres	No
DB_PASSWORD	Contraseña de PostgreSQL	Sin valor	Sí
DB_HOST_PORT	Puerto del host para PostgreSQL (solo 127.0.0.1)	5432	No
DB_URL	URL JDBC del backend fuera de Docker	jdbc:postgresql://localhost:5432/clinica_dental (en compose se arma con db:5432)	No
JWT_SECRET	Clave en base64 para firmar los JWT	Sin valor	Sí
JWT_EXPIRATION_MS	Duración del token	86400000	No
CORS_ALLOWED_ORIGINS	Orígenes permitidos por CORS	http://localhost:4200	No (en EC2 lo escribe el CD)
ADMIN_EMAIL / ADMIN_PASSWORD	Primer administrador, si no existe ningún ADMIN	Vacío	No, pero se requieren para el primer ingreso

Variable	Para qué sirve	Valor por defecto	Obligatoria
SMTP_HOST	Servidor de correo	Vacío (sin correos)	No
SMTP_PORT	Puerto SMTP	587	No
SMTP_USERNAME / SMTP_PASSWORD	Credenciales SMTP	Vacío	No
SMTP_FROM	Remitente (app.mail.remitente)	Vacío (sin correos)	No
RECORDATORIO_CRON	Horario del recordatorio (cron de Spring)	0 0 7 * * *	No
BACKEND_IMAGE / FRONTEND_IMAGE	Imagen a ejecutar	clinica-dental-backend:local / clinica-dental-frontend:local	No (en EC2, GHCR por SHA)
APP_VERSION_COMMIT / APP_VERSION_FECHA	Datos de GET /version	Los pone el Dockerfile con GIT_COMMIT y BUILD_DATE	No

Propiedades app.* de application.properties que gobiernan las reglas:

Propiedad	Valor	Uso
app.zona-horaria	America/Guatemala	Clock del backend y zona del cron
app.citas.duracion-minutos	30	Duración de cada cita
app.citas.antelacion-cancelacion-horas	24	Plazo del CLIENTE para cancelar y reprogramar
app.citas.max-inasistencias	3	Límite de NO_ASISTIO desde el último levantamiento
app.citas.recordatorio-cron	${RECORDATORIO_CRON:0 0 7 * * *}	Ejecución del recordatorio
app.jwt.expiration-ms	${JWT_EXPIRATION_MS:86400000}	Vigencia del token
app.cors.allowed-origins	${CORS_ALLOWED_ORIGINS:http://localhost:4200}	CORS

Propiedad	Valor	Uso
app.version.commit / fecha-construccion	${APP_VERSION_COMMIT:} / ${APP_VERSION_FECHA:}	GET /version
spring.jpa.hibernate.ddl-auto	validate	Valida el esquema al arrancar

En el servidor, el archivo .env (~/TEO1_ClinicaDental/.env) contiene DB_NAME, DB_USERNAME, DB_PASSWORD, JWT_SECRET, CORS_ALLOWED_ORIGINS, JWT_EXPIRATION_MS, BACKEND_IMAGE, FRONTEND_IMAGE, ADMIN_EMAIL, ADMIN_PASSWORD, SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD y SMTP_FROM. RECORDATORIO_CRON y DB_HOST_PORT no están definidas allí y toman su valor por defecto. Solo existen en GitHub Actions los secrets CI_DB_PASSWORD y CI_JWT_SECRET (CI), EC2_HOST, EC2_USERNAME y EC2_SSH_KEY (CD) y el GITHUB_TOKEN automático; el CD pasa al servidor DEPLOY_SHA, BACKEND_IMAGE, FRONTEND_IMAGE, CORS_ALLOWED_ORIGINS, GHCR_USERNAME y GHCR_TOKEN.

IMPORTANTE: los valores reales de las variables secretas no se escriben en el repositorio, en este manual ni en registros. JWT_SECRET puede generarse con openssl rand -base64 32.

10. Levantar el proyecto con Docker

10.1 Requisitos

Docker Engine con el plugin Compose (comando docker compose), Git y openssl. Sin Docker se necesitan JDK 21, Node.js 24.15.0 con npm 11 y PostgreSQL 16 con psql y createdb.

10.2 Pasos con Docker

1. Se clona el repositorio y se entra a su carpeta:

```
git clone https://github.com/TEO1Group/TEO1_ClinicaDental.git
cd TEO1_ClinicaDental
```

2. Se crea el archivo .env a partir de la plantilla y se genera la clave JWT:

```
cp .env.example .env
openssl rand -base64 32
```

3. En .env se llenan como mínimo DB_PASSWORD, JWT_SECRET (salida del comando anterior), ADMIN_EMAIL y ADMIN_PASSWORD. Las variables SMTP_* son opcionales.
4. Se construyen y levantan los servicios:

```
docker compose up --build
```

5. La primera vez que se crea el volumen postgres_data, el contenedor db carga db/clinica_dental_schema.sql.
6. Se verifica el acceso: frontend http://localhost:4200, backend http://localhost:8080, Swagger http://localhost:8080/swagger-ui.html o http://localhost:4200/api/swagger-ui.html. docker compose ps debe mostrar db como healthy.
7. Para detener: Ctrl+C en la terminal, o docker compose down si se levantó en segundo plano con -d.
8. Para reiniciar la base desde cero en desarrollo se borra el volumen y se vuelve a levantar; esto elimina todos los datos locales:

```
docker compose down -v
docker compose up --build
```

Si el puerto 5432 está ocupado se define en .env DB_HOST_PORT=5434 y DB_URL=jdbc:postgresql://localhost:5434/clinica_dental.

10.3 Levantar sin Docker (desarrollo)

Con PostgreSQL 16 en localhost:5432 se crea la base y se carga el esquema:

```
createdb -h localhost -U postgres clinica_dental
psql -h localhost -U postgres -d clinica_dental -v ON_ERROR_STOP=1 -f
db/clinica_dental_schema.sql
```

Backend, desde la raíz del repositorio y con el .env lleno:

```
set -a
. ./.env
set +a
cd backend
./mvnw spring-boot:run
```

Frontend, en otra terminal (npm start ejecuta ng serve con proxy.conf.json):

```
cd clinica-dental-frontend
npm ci
npm start
```

11. Pipeline de CI y CD

11.1 Integración continua (ci.yml)

Se dispara en push y pull_request hacia main o develop cuando cambian backend/**, db/**, clinica-dental-frontend/**, scripts/deploy-ec2.sh, docker-compose.yml, README.md, ci.yml o cd.yml. El job build-and-test corre en ubuntu-latest con un servicio postgres:16-alpine (base clinica_dental_ci, usuario ci_user) y ejecuta:**

1. Checkout y validación de que existan los secrets CI_DB_PASSWORD y CI_JWT_SECRET.

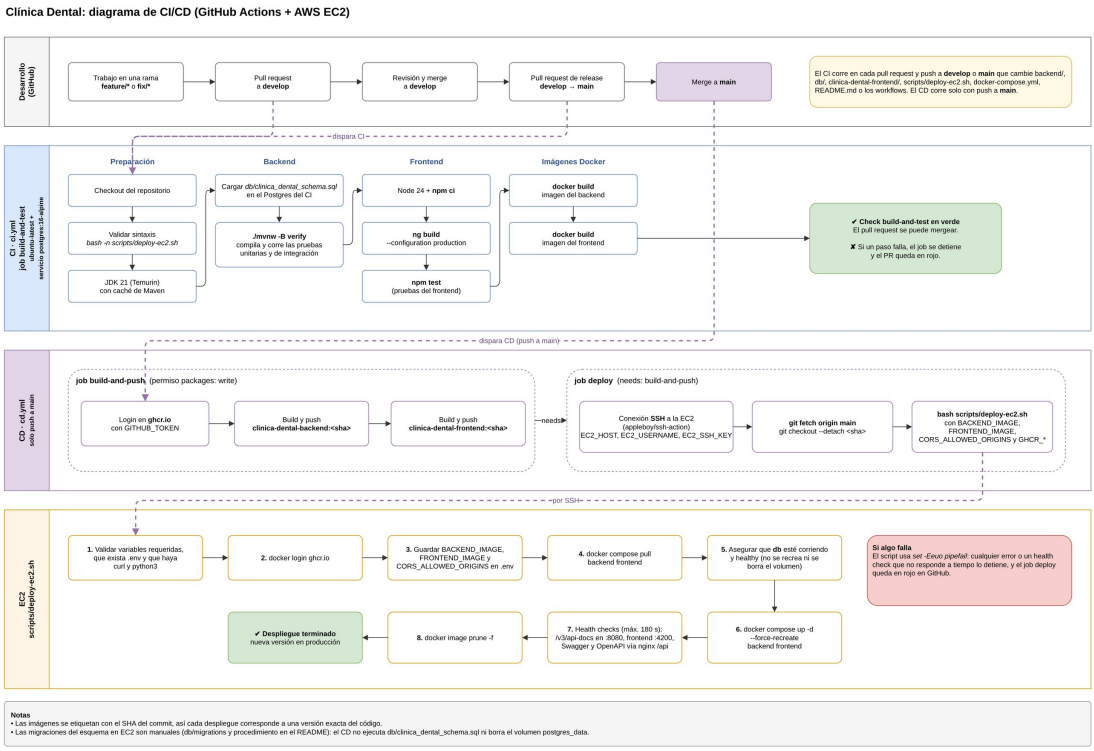
2. bash -n scripts/deploy-ec2.sh para validar la sintaxis del script de despliegue.
3. JDK 21 (temurin) con caché de Maven y carga de db/clinica_dental_schema.sql con psql.
4. ./mvnw -B verify en backend con las variables de la base de CI y SMTP vacío.
5. Node.js 24.15.0, npm ci, npm run build -- --configuration production y npm test -- --no-watch --no-progress.
6. docker build de las imágenes del backend y del frontend.

11.2 Despliegue continuo (cd.yml)

Se dispara con workflow_run cuando el workflow CI termina en la rama main. El job build-and-push solo corre si la conclusión es success, el evento original fue push, la rama es main y el repositorio es el mismo. Usa un grupo de concurrencia cd-main que no cancela despliegues en curso.

1. Hace checkout del SHA validado (workflow_run.head_sha) y rechaza la ejecución si ese SHA ya no es la punta de main.
2. Inicia sesión en ghcr.io con GITHUB_TOKEN y construye y publica ghcr.io/<owner>/clinica-dental-backend:<SHA> y ghcr.io/<owner>/clinica-dental-frontend:<SHA>, pasando GIT_COMMIT y BUILD_DATE al backend.
3. El job deploy se conecta por SSH (appleboy/ssh-action con EC2_HOST, EC2_USERNAME y EC2_SSH_KEY), entra a ~/TEO1_ClinicaDental, vuelve a comprobar que origin/main sea el SHA, hace checkout de ese commit y ejecuta scripts/deploy-ec2.sh.

scripts/deploy-ec2.sh valida variables y el .env, inicia sesión en GHCR, escribe BACKEND_IMAGE, FRONTEND_IMAGE y CORS_ALLOWED_ORIGINS en .env, descarga las imágenes, asegura que db esté running healthy, recrea solo backend y frontend sin build y espera hasta 180 segundos a que respondan /v3/api-docs (OpenAPI válido con título Clínica Dental API y versión 1.0.0), el frontend, Swagger UI y OpenAPI a través de nginx. Al final ejecuta docker image prune -f. El CD no aplica migraciones.

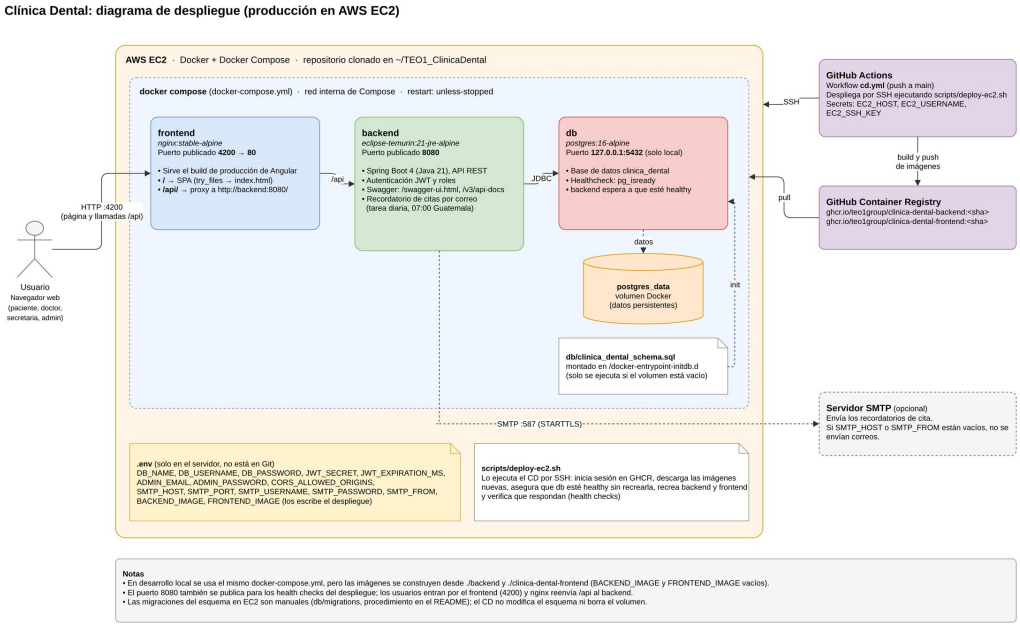


11.3 Flujo de ramas y protecciones

Cada tarea se desarrolla en una rama feature/, fix/, docs/ o codex/ y se integra a develop por pull request; cada release se integra de develop a main por pull request, lo que dispara el CD. Las reglas de protección configuradas en GitHub (consultadas con gh api) son iguales en main y develop:

Regla	main	develop
Cambios solo por pull request	Sí	Sí
Aprobaciones requeridas	1	1
Un push nuevo descarta la aprobación anterior	Sí	Sí
Conversaciones del PR resueltas	Sí	Sí
Push forzado	Bloqueado	Bloqueado
Borrar la rama	Bloqueado	Bloqueado
Historial lineal obligatorio	No	No
Commits firmados	No	No
Las reglas también aplican a los administradores	No	No
Checks de estado obligatorios	No	No

IMPORTANTE: no hay checks de estado obligatorios: el CI corre en cada pull request, pero GitHub no impide fusionar si falla; que el CI esté en verde es una práctica del equipo y no una regla configurada. Las reglas tampoco aplican a los administradores, que pueden saltarlas. Lo que sí es automático es que el CD solo corre después de un CI exitoso en main.



12. Despliegue en AWS EC2

12.1 Características de la instancia

Dato	Valor
Sistema operativo	Amazon Linux 2023 (Amazon Linux 2023.12.20260817), kernel 6.18.41-94.142.amzn2023.x86_64
Tipo de instancia	t3.micro
Región y zona	us-east-2, us-east-2b
CPU	2 vCPU
Memoria	913 MiB en total (174 MiB disponibles al tomar el dato)
Disco	/dev/nvme0n1p1 de 8.0 GB, 4.9 GB usados (62 %)
Docker / Compose	Docker 25.0.14 / Docker Compose v5.5.0
Arranque de Docker	Servicio habilitado al iniciar el sistema (enabled)
Grupo de seguridad	launch-wizard-1 (consola de AWS): 6 reglas de entrada IPv4 TCP con origen 0.0.0.0/0 para los puertos 22 (SSH), 80 (HTTP), 443

Dato	Valor
	(HTTPS), 4200, 8080 y 9000; 1 regla de salida. El puerto 5432 no está abierto. De esos puertos solo 22, 4200 y 8080 tienen un servicio escuchando; 80, 443 y 9000 están abiertos sin servicio (sección 5.6).
Usuario SSH	ec2-user
Puertos en escucha	22 (SSH), 4200 (frontend), 8080 (backend) en todas las interfaces; 5432 solo en 127.0.0.1
Commit desplegado	dc45780 (2026-10-28, Merge pull request #244)
Tareas programadas	Ninguna (sin crontab)

Contenedor	Imagen	Puertos
teo1_clinicadental-frontend-1	ghcr.io/teo1group/clinica-dental-frontend:dc457805...	0.0.0.0:4200->80
teo1_clinicadental-backend-1	ghcr.io/teo1group/clinica-dental-backend:dc457805...	0.0.0.0:8080->8080
teo1_clinicadental-db-1	postgres:16-alpine (healthy)	127.0.0.1:5432->5432

El volumen de datos es teo1_clinicadental_postgres_data (driver local). En el servidor quedan las imágenes de backend y frontend de los releases anteriores etiquetadas por SHA: dc457805, 34c8a6b6, 528324ad, 3a957365, 34910d09, 653663e6 y e81c7e37.

12.2 Estructura en el servidor

```
/home/ec2-user/
|-- TEO1_ClinicaDental/          repositorio en el commit desplegado
|   |-- .env                     variables reales (no versionado)
|   |-- docker-compose.yml
|   |-- db/ (schema, migrations, demo)
|   `-- scripts/ (deploy-ec2.sh, respaldar-db.sh, restaurar-db.sh)
|-- respaldos/                   respaldos de respaldar-db.sh
`-- clinica_dental_backups/      respaldos anteriores
```

12.3 Despliegue automático

Cada integración a main que pasa el CI publica las imágenes con el SHA y ejecuta scripts/deploy-ec2.sh por SSH (sección 11.2). El servidor solo descarga imágenes: no compila código.

12.4 Conexión SSH, logs y reinicio de servicios

```
ssh -i <ruta-de-la-clave> ec2-user@<IP-EC2>
cd ~/TEO1_ClinicaDental
docker compose ps
docker compose logs --tail 100 backend
docker compose logs -f frontend
docker compose restart backend
git log --oneline -1
```

IMPORTANTE: la clave privada nunca está en el repositorio; se guarda fuera de él con permisos 600 y solo la tienen las personas autorizadas. En GitHub vive como el secret EC2_SSH_KEY.

12.5 Migración manual antes de un release

El backend arranca con ddl-auto=validate: no crea ni altera tablas y no inicia si una entidad no coincide con la base. Por eso las migraciones se aplican a mano antes de integrar a main, en este orden:

1. Respaldo validado de la base (sección 13).
2. Inspección previa: docker compose ps db y, por ejemplo, la consulta de citas duplicadas del README antes del índice único.
3. Aplicación de cada archivo de db/migrations en orden (sección 4.2) con ON_ERROR_STOP:

```
docker compose exec -T db sh -lc \
  'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -v ON_ERROR_STOP=1' \
  < db/migrations/<archivo>.sql
```

4. Verificación con \d+ de las tablas cambiadas y conteos comparados con la inspección previa.
5. Integración a main; el CD despliega y hace sus comprobaciones de salud.

IMPORTANTE: nunca se ejecuta db/clinica_dental_schema.sql sobre EC2: empieza con DROP TABLE.

12.6 Comprobación posterior y datos de demostración

```
curl -s -o /dev/null -w "%{http_code}\n" localhost:8080/v3/api-docs
curl -s localhost:8080/version
docker compose ps
```

GET /version existe en origin/develop (VersionController.java:29) y es público (SecurityConfig.java:61). Llegó con el PR #268, que no forma parte del commit dc45780 (2026-10-28, Merge pull request #244), el que figura como desplegado en la información de la instancia; responde solo en una versión que incluya ese endpoint. En el navegador se revisa http://18.226.37.148:4200 y su Swagger. Los datos de demostración se cargan con el comando del README que pide la contraseña sin mostrarla y la pasa como clave_demo a psql dentro del contenedor db; solo con autorización del equipo y después de un respaldo.

12.7 Volver a una versión anterior

Como las imágenes anteriores quedan en el servidor con su SHA, se puede volver a una versión previa sin compilar:

1. Se guarda una copia privada del .env actual fuera del repositorio.
2. En .env se cambian BACKEND_IMAGE y FRONTEND_IMAGE al SHA anterior (por ejemplo el de 34c8a6b6...).
3. Se recrean solo backend y frontend:

```
docker compose up -d --no-build --no-deps --force-recreate backend frontend
docker compose ps
```

4. Se comprueban Swagger, la interfaz y el acceso a la base. Si la versión anterior esperaba un esquema distinto, el backend no arrancará por ddl-auto=validate y habrá que evaluar la base por separado.

13. Respaldo y restauración

Script	Qué hace
scripts/respaldar-db.sh [directorio]	pg_dump en formato custom a <directorio>/<base>-AAAA-MM-DD-HHMM.dump (por defecto ~/respaldos, permisos 600), valida con pg_restore --list y guarda un .conteos.txt con las filas por tabla.
scripts/restaurar-db.sh <archivo.dump> [base] [--reemplazar]	Restaura en una base nueva (por defecto clinica_dental_restore_test), nunca en la que usa el backend, y compara las filas con el .conteos.txt.

Ambos entran al contenedor con docker compose exec -T db (variable DB_EXEC) y usan las credenciales del contenedor, por lo que no piden ni escriben contraseñas.

```
cd ~/TEO1_ClinicaDental
scripts/respaldar-db.sh
scripts/restaurar-db.sh ~/respaldos/clinica_dental-AAAA-MM-DD-HHMM.dump
```

Para guardar una copia fuera de EC2 desde la computadora local:

```
scp -i <ruta-de-la-clave> 'ec2-user@<IP-EC2>:respaldos/clinica_dental-AAAA-MM-DD-HHMM.*' ~/respaldos-clinica/
```

Para recuperar producción, docs/respaldo-y-restauracion.md indica restaurar en clinica_dental_recuperada, detener el backend, renombrar la base actual a clinica_dental_anterior y la recuperada a clinica_dental, levantar el backend y verificar. En el servidor existen las carpetas /home/ec2-user/respaldos y /home/ec2-user/clinica_dental_backups. No hay tareas programadas: los respaldos se hacen a mano, siempre antes de una migración y de cada release.

IMPORTANTE: los respaldos contienen datos personales de pacientes; no se suben al repositorio ni a carpetas compartidas.

14. Pruebas

En origin/develop el backend tiene 33 clases de prueba en backend/src/test/java con 286 métodos de prueba: 285 anotados con @Test y 1 con @ParameterizedTest (conteo por lectura del código):

Tipo	Clases	Métodos	Ejemplos
Integración (@SpringBootTest con MockMvc y PostgreSQL)	21	134	CitasIntegrationTests (15), PlanesTratamientoIntegrationTests (16), PacientesDoctoresIntegrationTests (13), TratamientosIntegrationTests (11), OpenApiDocumentationIntegrationTests, SeguridadSprint5/6IntegrationTests, CitasConcurrencyIntegrationTests, FlujoCompletoIntegrationTests
Unitarias (servicios con dobles de prueba y Clock fijo)	12	152	CitaServiceTests (53), TratamientoServiceTests (18), ReprogramacionCitaServiceTests (17), ReporteCitaServiceTests (17), BloqueoDoctorServiceTests (16), DisponibilidadCitaServiceTests (13), JwtAuthenticationFilterTests (5, uno parametrizado), VersionControllerTests (4)
Frontend (Vitest)	1	2	src/app/app.spec.ts

La mayoría de las pruebas de integración se revierten con @Transactional. CitasConcurrencyIntegrationTests y AbonosConcurrencyIntegrationTests escriben datos reales y los borran al final, por lo que se usan solo en una base de desarrollo.

```
docker compose up -d db
set -a
. ./.env
set +a
cd backend
./mvnw verify
cd ..
cd clinica-dental-frontend
npm ci
npm test -- --no-watch --no-progress
npm run build -- --configuration production
```

En el CI se ejecutan las mismas pruebas (sección 11.1): ./mvnw -B verify contra el PostgreSQL de servicio con el esquema cargado y npm test del frontend; cualquier fallo detiene el pipeline y por lo tanto el CD.

15. Decisiones técnicas y justificación

Decisión	Justificación
ddl-auto=validate con esquema SQL y migraciones manuales	El esquema es la fuente de verdad; el backend falla al arrancar si no coincide, en lugar de crear tablas paralelas o alterar datos de producción sin control.
DTO de entrada y salida	Separan el contrato de la API de las entidades, permiten validar con anotaciones y evitan exponer campos internos como password_hash.
Borrado lógico (estado INACTIVO y tratamiento activo)	Se conserva la historia de citas, tratamientos, planes y abonos para reportes y auditoría.
Hora de Guatemala con un bean Clock	Todas las reglas de tiempo usan la misma zona sin depender del servidor, y las pruebas fijan el reloj.
Bloqueo pesimista del doctor y del plan más índice único parcial	Evita citas solapadas y abonos que superen el saldo cuando llegan peticiones simultáneas.
Costo copiado en cita_tratamiento	Un cambio de precio del catálogo no altera lo ya cobrado.
JWT sin estado	No hay sesiones en el servidor; el filtro igualmente rechaza usuarios inactivos.
nginx como proxy de /api	El navegador usa un solo origen y el backend no necesita exponerse para la interfaz.
Imágenes etiquetadas con el SHA en GHCR	Cada despliegue es trazable a un commit y se puede volver a una versión anterior sin compilar.
CD solo tras CI exitoso en main y verificación del SHA	Impide desplegar commits no probados u obsoletos.
Contratos de API en springdoc con prueba automática	OpenApiDocumentationIntegrationTests exige que cada operación tenga resumen, descripción, respuestas y seguridad.
Secretos solo en .env y GitHub Secrets	.env está en .gitignore y .env.example no tiene valores secretos.

16. Solución de problemas técnicos

Situación	Solución
El backend no arranca con un error de validación de esquema (Schema-validation)	Falta aplicar una migración de db/migrations. Se respalda la base y se aplican en orden (sección 12.5).
El backend no arranca por JWT_SECRET	La variable está vacía o no es base64 válido de al menos 32 bytes. Se genera con openssl rand -base64 32.
El backend no conecta a la base (password authentication failed)	DB_PASSWORD de .env no coincide con la del volumen existente; se corrige el valor o, solo en desarrollo, se recrea el volumen con docker compose down -v.
El navegador muestra errores de CORS	El origen no está en CORS_ALLOWED_ORIGINS. En local debe ser http://localhost:4200; en EC2 lo escribe el CD.
Puerto 5432, 8080 o 4200 ocupado	Se detiene el proceso que lo usa o, para PostgreSQL, se usa DB_HOST_PORT y DB_URL con otro puerto.
El CD no se ejecuta	Solo corre tras un CI exitoso de un push a main en este repositorio; se revisa el resultado del workflow CI.
El CD falla con 'El commit validado ya no es la punta de main'	Hubo un push posterior; el despliegue correcto es el del CI más reciente.
El CD falla esperando la salud de los servicios	Se revisan docker compose logs backend; suele ser una migración pendiente o una variable faltante.
EC2 no responde	Se comprueba el estado de la instancia en AWS, las reglas del grupo de seguridad y, por SSH, docker compose ps. Con 913 MiB de memoria, revisar también el uso de memoria.
No llegan los correos de recordatorio	SMTP_HOST o SMTP_FROM están vacíos, el cron aún no se ejecutó (07:00) o el envío falló; el log muestra 'No se pudo enviar el recordatorio'.
No se puede iniciar sesión como administrador	ADMIN_EMAIL y ADMIN_PASSWORD estaban vacíos en el primer arranque o ya existía un ADMIN; el log de AdminInitializer lo indica.
Las pruebas de integración fallan al conectar	Maven no lee .env: se exporta con set -a; . ./.env; set +a y se levanta db antes de ./mvnw verify.

Situación	Solución
El disco de EC2 se llena	Se eliminan imágenes antiguas que ya no se necesitan y respaldos copiados fuera del servidor.

17. Conclusiones

- El sistema sigue una arquitectura cliente-servidor en contenedores, con un backend en capas que separa API, reglas de negocio y acceso a datos, lo que facilita su mantenimiento y sus pruebas.
- Las reglas críticas (disponibilidad, plazos, inasistencias, saldos) están centralizadas en servicios, protegidas contra concurrencia y cubiertas por 286 pruebas automáticas del backend.
- La seguridad combina JWT, BCrypt, autorización por ruta y por dato y validación de entradas; quedan como mejoras el almacenamiento del token, el cierre del puerto 8080, HTTPS y la protección de Swagger.
- El pipeline de CI/CD despliega solo versiones probadas, trazables por SHA y reversibles, mientras que los cambios de esquema y los respaldos se mantienen como procedimientos manuales controlados.